```
const i18n = {
  zh: {
    title: "客製化靜觀瓶面板",
    subtitle: "選擇心情、顏色、形狀、音樂與圖像，立即生成你的靜觀瓶預覽。",
    controlsTitle: "控制面板",
    labelMood: "心情",
    labelColor: "顏色",
    labelShape: "形狀",
    labelDensity: "閃粉密度",
    labelImage: "上傳圖像",
    labelAudio: "上傳音樂",
    labelMusic: "背景音樂 / Background music",
    btnRender: "生成預覽 / Generate Preview",
    btnAnimStart: "開始動畫 / Start Animation",
    btnAnimStop: "停止動畫 / Stop Animation",
    note: "可作為開放日展示，也可再擴充成 PNG 或 GIF 匯出。",
    infoReady: "已生成預覽。",
    infoPlaying: "動畫進行中…",
    infoStopped: "已停止，畫面可作為 GIF 預覽。",
    moodPrefix: "心情",
    colorPrefix: "顏色",
    shapePrefix: "形狀",
    densityPrefix: "密度",
```

```
    imagePrefix: "圖像",
    musicOn: "已開啟背景音樂。",
    musicOff: "背景音樂已關閉。",
    imageLoaded: "已載入自訂圖像。",
    musicLoaded: "已載入自訂音樂。"
  },
  en: {
    title: "Custom Mindfulness Bottle Panel",
    subtitle: "Choose mood, color, shape, music, and image to generate a live bottle preview.",
    controlsTitle: "Control Panel",
    labelMood: "Mood",
    labelColor: "Color",
    labelShape: "Shape",
    labelDensity: "Glitter Density",
    labelImage: "Upload Image",
    labelAudio: "Upload Music",
    labelMusic: "Background music",
    btnRender: "Generate Preview",
    btnAnimStart: "Start Animation",
    btnAnimStop: "Stop Animation",
    note: "Suitable for open day demos and can be extended to PNG or GIF export.",
    infoReady: "Preview generated.",
    infoPlaying: "Animation running...",
```

```
        infoStopped: "Stopped. The frame can be used as a GIF preview.",
        moodPrefix: "Mood",
        colorPrefix: "Color",
        shapePrefix: "Shape",
        densityPrefix: "Density",
        imagePrefix: "Image",
        musicOn: "Background music enabled.",
        musicOff: "Background music disabled.",
        imageLoaded: "Custom image loaded.",
        musicLoaded: "Custom music loaded."
    }
};

const palette = {
    blue: ['#dff0ff', '#8fc6ff', '#2d75c6'],
    green: ['#e2f7eb', '#88d7a5', '#2b8f5d'],
    pink: ['#ffe3f0', '#ff9ec2', '#d34b82'],
    purple: ['#eee3ff', '#b796ff', '#7c4fd6'],
    teal: ['#dffcf7', '#7adfdb', '#1f9d96']
};

const moodEl = document.getElementById('mood');
const colorEl = document.getElementById('color');
const shapeEl = document.getElementById('shape');
```

```
const densityEl = document.getElementById('density');
const musicToggle = document.getElementById('musicToggle');
const imgUpload = document.getElementById('imgUpload');
const audioUpload = document.getElementById('audioUpload');
const bgm = document.getElementById('bgm');
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
const infoEl = document.getElementById('info');
const btnRender = document.getElementById('btnRender');
const btnAnim = document.getElementById('btnAnim');

let currentLang = 'zh';
let animating = false;
let raf = null;
let startTime = 0;
let particles = [];
let customImage = null;
const duration = 9000;

function t() { return i18n[currentLang]; }

function setLanguage(lang) {
    currentLang = lang;
    document.documentElement.lang = lang === 'zh' ? 'zh-Hant' : 'en';
```

```
    document.getElementById('title').textContent = i18n[lang].title;
    document.getElementById('subtitle').textContent = i18n[lang].subtitle;
    document.getElementById('controlsTitle').textContent = i18n[lang].controlsTitle;
    document.getElementById('labelMood').textContent = i18n[lang].labelMood;
    document.getElementById('labelColor').textContent = i18n[lang].labelColor;
    document.getElementById('labelShape').textContent = i18n[lang].labelShape;
    document.getElementById('labelDensity').textContent = i18n[lang].labelDensity;
    document.getElementById('labelImage').textContent = i18n[lang].labelImage;
    document.getElementById('labelAudio').textContent = i18n[lang].labelAudio;
    document.getElementById('labelMusic').textContent = i18n[lang].labelMusic;
    document.getElementById('btnRender').textContent = i18n[lang].btnRender;
    document.getElementById('btnAnim').textContent = animating ?
i18n[lang].btnAnimStop : i18n[lang].btnAnimStart;
    document.getElementById('note').textContent = i18n[lang].note;
    document.getElementById('btnZh').classList.toggle('active', lang === 'zh');
    document.getElementById('btnEn').classList.toggle('active', lang === 'en');
    render();
}

function pickTheme(mood) {
    const m = mood.toLowerCase();
    if (m.includes('焦慮') || m.includes('緊張') || m.includes('壓力') ||
m.includes('anx') || m.includes('stress') || m.includes('nerv')) return 'blue';
    if (m.includes('憤怒') || m.includes('生氣') || m.includes('嬲') ||
m.includes('angry') || m.includes('mad')) return 'green';
```

```
  if (m.includes('疲') || m.includes('乏') || m.includes('累') || m.includes('tired') || m.includes('sleep')) return 'pink';
  if (m.includes('開心') || m.includes('興奮') || m.includes('happy') || m.includes('excited')) return 'teal';
  if (m.includes('難過') || m.includes('失望') || m.includes('sad') || m.includes('down')) return 'purple';
  return 'teal';
}

function getTheme() {
  return colorEl.value === 'auto' ? pickTheme(moodEl.value) : colorEl.value;
}

function buildGradient(theme, x, y, w, h) {
  if (theme === 'rainbow') {
    const g = ctx.createLinearGradient(x, y, x + w, y);
    g.addColorStop(0, '#ff4d4d');
    g.addColorStop(0.16, '#ff9f43');
    g.addColorStop(0.33, '#ffd93d');
    g.addColorStop(0.5, '#7bed9f');
    g.addColorStop(0.66, '#70a1ff');
    g.addColorStop(0.83, '#a55eea');
    g.addColorStop(1, '#ff6b81');
    return g;
  }
```

```
    const [soft] = palette[theme];
    const g = ctx.createLinearGradient(x, y, x + w, y + h);
    g.addColorStop(0, 'rgba(255,255,255,.62)');
    g.addColorStop(.4, soft);
    g.addColorStop(1, 'rgba(255,255,255,.36)');
    return g;
}

function bottlePath(shape, x, y, w, h) {
    ctx.beginPath();
    if (shape === 'square') ctx.roundRect(x, y, w, h, 38);
    else if (shape === 'tube') ctx.roundRect(x + w * 0.18, y + 18, w * 0.64, h - 36, 88);
    else if (shape === 'heart') {
        const cx = x + w / 2, top = y + h * 0.28;
        ctx.moveTo(cx, y + h * 0.92);
        ctx.bezierCurveTo(x + w * 0.08, y + h * 0.7, x + w * 0.06, top, cx, top + 20);
        ctx.bezierCurveTo(x + w * 0.94, top, x + w * 0.92, y + h * 0.7, cx, y + h * 0.92);
    } else if (shape === 'tall') ctx.roundRect(x + w * 0.12, y, w * 0.76, h, 95);
    else ctx.roundRect(x, y, w, h, 130);
}

function initParticles(theme, densityLabel, shape) {
    const countMap = { low: 28, mid: 55, high: 92 };
    const count = countMap[densityLabel] || 55;
```

```
    const colors = theme === 'rainbow'
        ? ['#ff4d4d', '#ff9f43', '#ffd93d', '#7bed9f', '#70a1ff', '#a55eea', '#ff6b81']
        : palette[theme];

    particles = Array.from({ length: count }, () => ({
        x: 0,
        y: 0,
        r: 2 + Math.random() * 5,
        vy: 0.18 + Math.random() * 0.7,
        vx: (Math.random() - 0.5) * 0.35,
        c: colors[Math.floor(Math.random() * colors.length)],
        a: 0.5 + Math.random() * 0.38
    }));
    resetParticles(shape);
}

function resetParticles(shape) {
    const zones = {
        round: { x1: 225, x2: 675, y1: 170, y2: 690 },
        tall: { x1: 250, x2: 650, y1: 160, y2: 700 },
        square: { x1: 220, x2: 680, y1: 160, y2: 690 },
        heart: { x1: 255, x2: 645, y1: 190, y2: 650 },
        tube: { x1: 300, x2: 600, y1: 165, y2: 695 }
    }[shape] || { x1: 225, x2: 675, y1: 170, y2: 690 };
```

```
    particles.forEach(p => {
        p.x = zones.x1 + Math.random() * (zones.x2 - zones.x1);
        p.y = zones.y1 + Math.random() * (zones.y2 - zones.y1);
    });
}

function drawBackground() {
    const g = ctx.createLinearGradient(0, 0, 0, canvas.height);
    g.addColorStop(0, '#f8fcff');
    g.addColorStop(1, '#edf5ff');
    ctx.fillStyle = g;
    ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawImageLayer(shape, x, y, w, h) {
    if (!customImage) return;
    ctx.save();
    bottlePath(shape, x + 34, y + 70, w - 68, h - 150);
    ctx.clip();
    const imgW = w - 90, imgH = h - 190;
    const scale = Math.max(imgW / customImage.width, imgH / customImage.height);
    const dw = customImage.width * scale, dh = customImage.height * scale;
    const dx = x + (w - dw) / 2;
```

```
    const dy = y + 90 + ((h - 170) - dh) / 2;
    ctx.globalAlpha = 0.28;
    ctx.drawImage(customImage, dx, dy, dw, dh);
    ctx.globalAlpha = 1;
    ctx.restore();
}

function drawFrame(time) {
    const mood = moodEl.value;
    const theme = getTheme();
    const shape = shapeEl.value;
    const density = densityEl.value;
    const elapsed = time - startTime;
    const settling = animating ? Math.min(elapsed / duration, 1) * 0.55 : 0;
    const deep = theme === 'rainbow' ? '#ff6b81' : palette[theme][2];

    drawBackground();
    ctx.fillStyle = '#214';
    ctx.font = 'bold 34px Arial';
    ctx.fillText(currentLang === 'zh' ? '客製化靜觀瓶 / Custom Mindfulness Bottle' :
'Custom Mindfulness Bottle', 40, 54);
    ctx.font = '22px Arial';
    ctx.fillStyle = '#456';
    ctx.fillText(`${t().moodPrefix}: ${mood || '—'}`, 40, 92);
```

```
    ctx.fillText(`${t().colorPrefix}: ${theme} | ${t().shapePrefix}: ${shape} |
${t().densityPrefix}: ${density}`, 40, 126);

    const x = 160, y = 120, w = 580, h = 690;
    ctx.save();
    ctx.shadowColor = 'rgba(0,0,0,.11)';
    ctx.shadowBlur = 26;
    ctx.fillStyle = 'rgba(255,255,255,.34)';
    bottlePath(shape, x + 18, y + 12, w - 36, h - 24);
    ctx.fill();
    ctx.restore();

    ctx.fillStyle = buildGradient(theme, x, y, w, h);
    ctx.strokeStyle = 'rgba(120,150,180,.6)';
    ctx.lineWidth = 6;
    bottlePath(shape, x + 14, y + 8, w - 28, h - 16);
    ctx.fill();
    ctx.stroke();

    drawImageLayer(shape, x, y, w, h);

    ctx.fillStyle = '#d9e7f3';
    ctx.beginPath();
    ctx.roundRect(x + 190, y - 34, 200, 72, 18);
```

```
    ctx.fill();
    ctx.strokeStyle = 'rgba(120,150,180,.5)';
    ctx.stroke();

    ctx.fillStyle = 'rgba(255,255,255,.22)';
    ctx.beginPath();
    ctx.roundRect(x + 190, y + 48, 34, h - 120, 16);
    ctx.fill();

    const settleY = y + h * 0.64;
    const bottomGrad = ctx.createLinearGradient(0, settleY - 40, 0, settleY + 180);
    bottomGrad.addColorStop(0, 'rgba(255,255,255,0)');
    bottomGrad.addColorStop(1, 'rgba(255,255,255,.68)');
    ctx.fillStyle = bottomGrad;

    if (shape === 'heart') {
        ctx.beginPath();
        ctx.moveTo(x + w / 2, settleY + 230);
        ctx.bezierCurveTo(x + w * 0.12, settleY + 120, x + w * 0.08, settleY - 40, x + w / 2,
settleY + 10);
        ctx.bezierCurveTo(x + w * 0.92, settleY - 40, x + w * 0.88, settleY + 120, x + w / 2,
settleY + 230);
        ctx.fill();
    } else if (shape === 'tube') {
        ctx.beginPath();
```

```
    ctx.roundRect(x + 132, settleY, w - 264, h - (settleY - y) - 18, 90);
    ctx.fill();
} else {
    ctx.beginPath();
    ctx.roundRect(x + 52, settleY, w - 104, h - (settleY - y) - 18, 110);
    ctx.fill();
}

if (theme === 'rainbow') {
    const rg = ctx.createLinearGradient(x + 70, y + 180, x + w - 70, y + 180);
    rg.addColorStop(0, '#ff4d4d');
    rg.addColorStop(.16, '#ff9f43');
    rg.addColorStop(.33, '#ffd93d');
    rg.addColorStop(.5, '#7bed9f');
    rg.addColorStop(.66, '#70a1ff');
    rg.addColorStop(.83, '#a55eea');
    rg.addColorStop(1, '#ff6b81');
    ctx.fillStyle = rg;
    ctx.globalAlpha = 0.25;
    bottlePath(shape, x + 36, y + 72, w - 72, h - 150);
    ctx.fill();
    ctx.globalAlpha = 1;
}
```

```
particles.forEach(p => {
    p.y += p.vy + settling * 2.0;
    p.x += p.vx;
    if (p.y > 760) {
        p.y = y + 110 + Math.random() * 70;
        p.x = x + 80 + Math.random() * (w - 160);
    }
    ctx.globalAlpha = p.a;
    ctx.fillStyle = p.c;
    ctx.beginPath();
    ctx.arc(p.x, p.y, p.r, 0, Math.PI * 2);
    ctx.fill();
});
ctx.globalAlpha = 1;

for (let i = 0; i < 18; i++) {
    const px = x + 80 + Math.random() * (w - 160);
    const py = y + 150 + Math.random() * (h - 260);
    ctx.fillStyle = 'rgba(255,255,255,.75)';
    ctx.beginPath();
    ctx.arc(px, py, 1.4 + Math.random() * 2.8, 0, Math.PI * 2);
    ctx.fill();
}
```

```
    ctx.fillStyle = deep;
    ctx.font = 'bold 26px Arial';
    ctx.fillText(
        animating
            ? (currentLang === 'zh' ? '動畫進行中… 靜靜沉澱' : 'Animation running... settling gently')
            : (currentLang === 'zh' ? '按下「開始動畫」預覽瓶內流動' : 'Press Start Animation to preview movement'),
        40,
        170
    );
    ctx.fillStyle = '#476';
    ctx.font = '20px Arial';
    ctx.fillText(
        currentLang === 'zh'
            ? '可上傳圖像與音樂，打造學校專屬版本。'
            : 'Upload image and music to make a school-specific version.',
        40,
        202
    );

    const pct = Math.min(elapsed / duration, 1);
    let text = animating ? `${t().infoPlaying} ${Math.round(pct * 100)}%` : t().infoReady;
    text += `\n${t().moodPrefix}: ${mood || '（未輸入）'}`;
```

```
    text += `\n${t().colorPrefix}: ${theme} | ${t().shapePrefix}: ${shape} |
${t().densityPrefix}: ${density}`;
    if (customImage) text += `\n${t().imagePrefix}: ${currentLang === 'zh' ? '已載入自
訂圖像' : 'Loaded'}`;
    infoEl.textContent = text;

    if (animating && elapsed < duration) {
        raf = requestAnimationFrame(drawFrame);
    } else if (animating) {
        animating = false;
        btnAnim.textContent = t().btnAnimStart;
        raf = null;
        if (!musicToggle.checked) stopMusic();
    }
}

function render() {
    if (!particles.length) initParticles(getTheme(), densityEl.value, shapeEl.value);
    resetParticles(shapeEl.value);
    drawFrame(performance.now());
}

function startAnimation() {
    if (raf) cancelAnimationFrame(raf);
    initParticles(getTheme(), densityEl.value, shapeEl.value);
```

```
    animating = true;
    startTime = performance.now();
    btnAnim.textContent = t().btnAnimStop;
    if (musicToggle.checked) startMusic();
    raf = requestAnimationFrame(drawFrame);
}

function stopAnimation() {
    animating = false;
    if (raf) cancelAnimationFrame(raf);
    raf = null;
    btnAnim.textContent = t().btnAnimStart;
    drawFrame(performance.now());
}

let audioCtx = null;
let osc = null;
let gain = null;

function startMusic() {
    const theme = getTheme();
    if (audioCtx && audioCtx.state === 'suspended') audioCtx.resume();
    if (!audioCtx) {
        audioCtx = new (window.AudioContext || window.webkitAudioContext)();
```

```
        gain = audioCtx.createGain();
        gain.gain.value = 0.03;
        gain.connect(audioCtx.destination);
    }
    if (osc) return;
    osc = audioCtx.createOscillator();
    osc.type = 'sine';
    osc.frequency.value =
        theme === 'rainbow' ? 220 :
        theme === 'pink' ? 196 :
        theme === 'purple' ? 174 :
        theme === 'green' ? 146 : 164;
    osc.connect(gain);
    osc.start();
    infoEl.textContent += `\n${t().musicOn}`;
}

function stopMusic() {
    if (osc) {
        try { osc.stop(); } catch (e) {}
        osc.disconnect();
        osc = null;
    }
    infoEl.textContent += `\n${t().musicOff}`;
```

```
}

imgUpload.addEventListener('change', (e) => {
    const file = e.target.files && e.target.files[0];
    if (!file) return;
    const url = URL.createObjectURL(file);
    const img = new Image();
    img.onload = () => {
        customImage = img;
        render();
    };
    img.src = url;
});

audioUpload.addEventListener('change', (e) => {
    const file = e.target.files && e.target.files[0];
    if (!file) return;
    const url = URL.createObjectURL(file);
    bgm.src = url;
    if (musicToggle.checked) bgm.play().catch(() => {});
    infoEl.textContent = `${infoEl.textContent}\n${t().musicLoaded}`;
});

btnRender.addEventListener('click', render);
```

```
btnAnim.addEventListener('click', () => animating ? stopAnimation() :
startAnimation());


['change', 'input'].forEach(ev => {
    moodEl.addEventListener(ev, render);
    colorEl.addEventListener(ev, () => {
        render();
        if (musicToggle.checked && animating) {
            stopMusic();
            startMusic();
        }
    });
    shapeEl.addEventListener(ev, render);
    densityEl.addEventListener(ev, render);
});


musicToggle.addEventListener('change', () => {
    if (musicToggle.checked) {
        if (bgm.src) bgm.play().catch(() => {});
        if (animating) startMusic();
    } else {
        bgm.pause();
        stopMusic();
    }
```

```
    render();
});

document.getElementById('btnZh').addEventListener('click', () => setLanguage('zh'));
document.getElementById('btnEn').addEventListener('click', () => setLanguage('en'));

setLanguage('zh');
render();
```